arXiv:submit/7377958 [cs.AI] 17 Mar 2026

Path-Asymmetric Low-Rank Adaptation of Hybrid State Space–Transformer Models: An Empirical Study on Falcon H1 With Cross-Domain Evaluation on Language Modelling and Function-Calling Tasks

Research Report
Bilel CHERIF
bilel.cherif@tii.ae

March 17, 2026

Abstract

Parameter-efficient fine-tuning of large language models has converged on Low-Rank Adaptation (LoRA) as the *de facto* standard, yet its application to hybrid architectures that interleave State Space Models (SSMs) with Transformer attention remains largely unexplored. Such models expose two structurally distinct computational pathways — a Mamba-style SSM path with linear-time recurrent complexity and a multi-head attention path with quadratic complexity — whose inductive biases and gradient dynamics during fine-tuning fundamentally differ.

In this work, we investigate *path-asymmetric LoRA*: a fine-tuning strategy that assigns differentiated adapter ranks to the SSM and attention paths according to their relative gradient signal, rather than applying a uniform rank across all linear layers. We conduct a controlled empirical study on **Falcon H1-1B** [1], a hybrid Mamba-2–Transformer model, evaluating three LoRA configurations across two semantically distinct benchmarks: next-token prediction on WikiText-2 [2] and structured function calling on the xLAM benchmark [3].

Direct inspection of the Falcon H1-1B checkpoint reveals two practical constraints absent from prior hybrid fine-tuning literature: (i) the model employs additive path combination rather than a learnable gate, invalidating gate-adaptation approaches; and (ii) the PEFT library enforces a leaf-name blacklist that blocks `mamba.out_proj` adaptation for `model_type=falcon_h1`.

Our principal contributions are as follows. **(1) Empirical validation of path asymmetry for language modelling:** path-asymmetric LoRA with r_{ssm}=16, r_{attn}=4 achieves perplexity 6.74 on WikiText-2 using only 4.39M trainable parameters — a 3.7× reduction relative to symmetric LoRA (16.24M parameters, perplexity 6.84) at identical wall-clock training time. **(2) Domain-reversal finding:** on the xLAM function-calling task, the *inverted* configuration (r_{ssm}=4, r_{attn}=16) outperforms all other strategies on every metric, achieving Full-EM 0.275 versus 0.220 for standard asymmetric LoRA — a 25% relative gain. **(3) Rank-modulation of ρ:** we show that the gradient norm ratio $\rho = \|\nabla_{\Theta_m}\mathcal{L}\|_F / \|\nabla_{\Theta_a}\mathcal{L}\|_F$, commonly proposed as a predictor of optimal rank allocation, is itself a function of the rank allocation, shifting from ρ=8.45 (asymmetric) to ρ=0.80 (inverted), rendering it unreliable as a pre-hoc selection criterion.

These results motivate a practical recommendation: always ablate both rank directions before a full fine-tuning run, as the optimal direction is task-dependent and cannot be predicted from gradient statistics alone.

This work have been conducted independently from the language model team and does not reflect any opinions or endorcement of TII. Most of the experiments have been conducted with very limited ressources which might lead to different results at full scale.

Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Research Questions . . . 3
1.3 Contributions . . . 3
1.4 Paper Organisation . . . 4

2 Background and Related Work **4**
2.1 State Space Models . . . 4
2.2 Multi-Head Attention . . . 5
2.3 Low-Rank Adaptation (LoRA) . . . 5
2.4 Hybrid SSM-Transformer Models . . . 6

3 The Falcon H1 Architecture **6**
3.1 Layer Structure from Direct Inspection . . . 6
3.2 Hybrid Layer Formulation . . . 6

4 Methods: Path-Asymmetric LoRA **7**
4.1 Notation . . . 7
4.2 Strategy S1: Symmetric LoRA (Baseline) . . . 7
4.3 Strategy S2: Path-Asymmetric LoRA . . . 8
4.4 Strategy S3: Inverted Asymmetric LoRA (Ablation) . . . 8
4.5 Gradient Norm Ratio . . . 8
4.6 Path-Asymmetric LoRA Fine-Tuning algorithm . . . 8

5 Experimental Setup **10**
5.1 Model . . . 10
5.2 Datasets . . . 10
5.3 Hyperparameters . . . 10
5.4 Evaluation Metrics . . . 10

6 Language Modelling Results **11**
6.1 Trainable Parameters . . . 11
6.2 Perplexity and Gradient Analysis . . . 12

7 Tool-Use Results **12**
7.1 Results . . . 12
7.2 Key Observations . . . 12

8 Analysis **13**
8.1 Task-Dependent Rank Allocation . . . 13
8.2 ρ as a Rank-Modulated Statistic . . . 14
8.3 Symmetric LoRA and Uniform Rank Waste . . . 14
8.4 Practical Implications . . . 15

9 Conclusion **15**

A Fine-Tuning Templates **17**
A.1 Language Modelling Template (WikiText-2) . . . 17
A.2 Tool-Use Template (xLAM) . . . 18

1 Introduction

1.1 Motivation

The dominant paradigm for adapting large pre-trained language models to downstream tasks has shifted decisively toward parameter-efficient fine-tuning (PEFT) methods, with Low-Rank Adaptation [LoRA; 4] emerging as the most widely adopted approach. LoRA constrains weight updates to a low-rank subspace, reducing trainable parameter counts from billions to millions while preserving competitive task performance. Subsequent work has demonstrated that LoRA can be combined with quantisation [QLoRA; 5], extended to capture higher-order interactions [LoRA+; 6], and improved through adaptive rank selection [AdaLoRA; 7].

Concurrently, a new class of *hybrid* sequence models has emerged that combines the computational efficiency of State Space Models [SSMs; 8] with the representational power of Transformer attention [9]. Models such as Jamba [10], Zamba [11], and Falcon H1 [1] interleave SSM and attention blocks at various ratios, achieving favourable perplexity–efficiency trade-offs while maintaining sub-quadratic inference complexity. The Mamba architecture [12], which introduces input-dependent (selective) state transitions, has become the dominant SSM component in such hybrid models.

Despite the independent maturation of these two research directions, their intersection remains underexplored. Prior work on LoRA for hybrid models has either treated all linear layers uniformly [13] or focused exclusively on pure SSM architectures [14]. The structural asymmetry between SSM and attention pathways — their distinct parameterisations, inductive biases, and computational complexities — suggests that uniform rank allocation may be sub-optimal and that architecture-aware rank differentiation could yield both efficiency and performance gains.

1.2 Research Questions

This work addresses three specific questions:

1. **Path asymmetry on language modelling.** Does concentrating LoRA capacity on the SSM path ($r_{\text{ssm}} > r_{\text{attn}}$) improve parameter efficiency on next-token prediction, consistent with the hypothesis that Mamba's local sequential processing is the primary adaptation target for language modelling?

2. **Rank-modulation of gradient statistics.** Can the gradient norm ratio ρ between paths serve as a reliable, architecture-fixed predictor of optimal rank direction, or is ρ itself a function of the rank allocation?

3. **Task-dependence of optimal direction.** Does the optimal rank direction generalise across semantically distinct tasks, or does a task that stresses the attention path's schema-tracking capacity (structured function calling) reverse the finding from language modelling?

1.3 Contributions

We make the following contributions:

1. We introduce and formalise *path-asymmetric LoRA* for hybrid SSM-Transformer models, providing a mathematical treatment and its parameter complexity analysis.

2. We conduct the first controlled rank-direction ablation on Falcon H1, a production hybrid model, across two evaluation domains with identical hyperparameters.

3. We document two previously unreported constraints for Falcon H1 fine-tuning: the absence of a learnable gate and the PEFT leaf-name blacklist affecting `mamba.out_proj`.

4. We show empirically that ρ is rank-modulated, providing a concrete negative result on its use as a selection criterion.
5. We provide all experimental code, configurations, and evaluation templates to facilitate reproducibility.

1.4 Paper Organisation

Section 2 reviews the relevant background on SSMs, Transformer attention, hybrid architectures, and LoRA. Section 3 characterises the Falcon H1 architecture from direct inspection. Section 4 formalises the three fine-tuning strategies. Section 5 describes the experimental setup. Sections 6 and 7 report language modelling and tool-use results respectively. Section 8 provides unified analysis. Section 9 concludes. Appendix A documents fine-tuning templates.

2 Background and Related Work

2.1 State Space Models

Structured State Space Models [S4; 8] model sequences via a latent state recurrence. Given an input $x \in \mathbb{R}^L$ and a state dimension N, the continuous-time SSM is:

$$\dot{h}(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t), \tag{1}$$

where $A \in \mathbb{R}^{N\times N}$, $B \in \mathbb{R}^{N\times 1}$, $C \in \mathbb{R}^{1\times N}$.

Under zero-order hold discretisation with step size $\Delta > 0$:

$$\bar{A} = e^{\Delta A}, \quad \bar{B} = (\Delta A)^{-1}(e^{\Delta A} - I)\,\Delta B, \tag{2}$$

yielding the recurrent form:

$$h_t = \bar{A}\,h_{t-1} + \bar{B}\,x_t, \tag{3}$$
$$y_t = C\,h_t + D\,x_t, \tag{4}$$

which admits an equivalent convolution representation enabling parallel training:

$$y = \bar{K} * x, \quad \bar{K} = (C\bar{B},\ C\bar{A}\bar{B},\ \ldots,\ C\bar{A}^{L-1}\bar{B}). \tag{5}$$

Mamba. Gu and Dao [12] introduce *selective* state spaces by making Δ, B, and C functions of the input:

$$(\Delta_t,\, B_t,\, C_t) = \mathrm{Linear}(x_t), \tag{6}$$

enabling data-dependent state transitions. This selectivity allows the model to filter irrelevant information and remember long-range dependencies selectively.

The full Mamba block for a d-dimensional input $\mathbf{x} \in \mathbb{R}^{L\times d}$ is:

$$z,\ x' = \mathrm{split}\big(W_{\mathrm{in}}\,\mathbf{x}\big), \tag{7}$$
$$\tilde{x}' = \mathrm{SiLU}\big(\mathrm{Conv1d}(x')\big), \tag{8}$$
$$\mathbf{y}_{\mathrm{ssm}} = \mathrm{SSM}(\tilde{x}'), \tag{9}$$
$$\mathbf{y}_{\mathrm{mamba}} = W_{\mathrm{out}}\big(\mathbf{y}_{\mathrm{ssm}} \odot \sigma(z)\big), \tag{10}$$

where $W_{\mathrm{in}} \in \mathbb{R}^{2Ed\times d}$ (expansion factor E), $W_{\mathrm{out}} \in \mathbb{R}^{d\times Ed}$, and σ is the SiLU activation.

Training vs. inference complexity. During inference, Mamba uses Eq. (3): $\mathcal{O}(1)$ per step and 5× higher throughput than Transformers of comparable size [12]. During *training*, Mamba uses the parallel convolutional form Eq. (5), whose throughput is comparable to or *lower* than FlashAttention [13]. This distinction is critical: fine-tuning speed advantages attributed to SSMs are an inference-time phenomenon and do not carry over to the training backward pass.

2.2 Multi-Head Attention

Given a sequence $\mathbf{X} \in \mathbb{R}^{L\times d}$, scaled dot-product attention computes:

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{11}$$

where $Q = \mathbf{X}W^Q$, $K = \mathbf{X}W^K$, $V = \mathbf{X}W^V$, with projection matrices $W^Q, W^K \in \mathbb{R}^{d\times d_k}$ and $W^V \in \mathbb{R}^{d\times d_v}$. Multi-head attention runs H such operations in parallel:

$$\mathrm{MHA}(\mathbf{X}) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H)W^O, \tag{12}$$

$\mathrm{head}_i = \mathrm{Attn}(Q_i, K_i, V_i)$.

The $QK^\top$ matrix requires $\mathcal{O}(L^2)$ memory during backpropagation — the dominant memory cost in fine-tuning at sequence lengths beyond 512 tokens.

2.3 Low-Rank Adaptation (LoRA)

Hu et al. [4] observe that weight updates for pre-trained models have a low intrinsic rank. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$, LoRA constrains the update:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r}\, B\, A, \tag{13}$$

where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, $r \ll \min(d, k)$, and $\alpha > 0$ is a scaling hyperparameter. At initialisation, $A \sim \mathcal{N}(0, \sigma^2)$ and $B = 0$, so $\Delta W = 0$ at the start of training.

The modified forward pass is:

$$h = W_0\, x + \frac{\alpha}{r}\, B\, A\, x = h_0 + \frac{\alpha}{r}\, B\, (Ax), \tag{14}$$

where W_0 is frozen. Trainable parameter count: $r(d + k)$ vs. dk for full fine-tuning.

QLoRA. Dettmers et al. [5] extend LoRA to 4-bit NF4-quantised base weights, enabling fine-tuning of billion-parameter models on single consumer GPUs:

$$h \approx \hat{W}_0\, x + \frac{\alpha}{r}\, B\, A\, x, \quad \hat{W}_0 = \mathrm{NF4}(W_0), \tag{15}$$

where $\hat{W}_0$ is dequantised during the forward pass and only A, B are stored in full precision.

PEFT for SSMs. Galim et al. [13] benchmark LoRA and other PEFT methods on Mamba and Jamba, finding that LoRA on linear projection matrices outperforms alternatives. Crucially, for Jamba (a sequential hybrid), all PEFT methods are applied to Mamba layers while Transformer layers are frozen, without investigating whether the rank allocation between paths is optimal. Halloran et al. [14] propose Lyapunov-regularised LoRA specifically for pure Mamba fine-tuning, targeting the large memory buffers of Mamba's CUDA kernels. Neither work studies *parallel-path* hybrid architectures or the rank-direction question.

2.4 Hybrid SSM-Transformer Models

Several recent architectures combine SSM and attention blocks in various configurations:

- **Sequential interleaving** (Jamba [10], Zamba [11]): alternating SSM and attention layers.
- **Parallel combination** (Falcon H1 [1], Hymba [15]): SSM and attention paths process the same input simultaneously, with outputs combined per layer.

The parallel combination is architecturally distinct: every token passes through both pathways at every layer, and the outputs are merged before the residual addition. This structure is the focus of the present study.

3 The Falcon H1 Architecture

3.1 Layer Structure from Direct Inspection

Falcon H1-1B consists of 24 hybrid layers, each processing input $\mathbf{x}^{(\ell)} \in \mathbb{R}^{L\times d}$ with $d = 2048$ through two parallel pathways followed by an additive residual combination. Direct inspection of the checkpoint reveals the linear layer structure in Table 1.

Table 1: Linear layers per hybrid layer of Falcon H1-1B (d=2048, 24 layers). The PEFT column reports whether the layer's leaf name is blocked by `peft` for `model_type=falcon_h1`. Blocked layers raise a `ValueError` if included in `target_modules`.

Sub-module	Layer name	d_{in}	d_{out}	PEFT
`mamba`	`in_proj`	2048	6704	Allowed
	`out_proj`	3072	2048	**Blocked**
`self_attn` (GQA)	`q_proj`	2048	1024	Allowed
	`k_proj`	2048	256	Allowed
	`v_proj`	2048	256	Allowed
	`o_proj`	1024	2048	Allowed
`feed_forward` (SwiGLU)	`gate_proj`	2048	4608	Allowed
	`up_proj`	2048	4608	Allowed
	`down_proj`	4608	2048	Allowed

3.2 Hybrid Layer Formulation

Let $\mathbf{x}^{(\ell)} \in \mathbb{R}^{L\times d}$ denote the input to layer ℓ. The SSM path produces:

$$\mathbf{m}^{(\ell)} = W_{\text{out}}^{(\ell)} \cdot \text{Mamba}\left(W_{\text{in}}^{(\ell)}\,\mathbf{x}^{(\ell)}\right), \tag{16}$$

where $W_{\text{in}}^{(\ell)} \in \mathbb{R}^{6704\times 2048}$ and $W_{\text{out}}^{(\ell)} \in \mathbb{R}^{2048\times 3072}$. The attention path produces:

$$\mathbf{a}^{(\ell)} = W_o^{(\ell)} \cdot \text{MHA}\left(\mathbf{x}^{(\ell)}; W^{Q,(\ell)}, W^{K,(\ell)}, W^{V,(\ell)}\right), \tag{17}$$

where $W_o^{(\ell)} \in \mathbb{R}^{2048\times 1024}$.

The layer output is:

$$\mathbf{x}^{(\ell+1)} = \mathbf{m}^{(\ell)} + \mathbf{a}^{(\ell)} + \mathbf{x}^{(\ell)}. \tag{18}$$

Remark 1 (No learned gate)**.** *Equation (18) confirms that Falcon H1 combines the two path outputs by* addition*, not by a learned softmax gate* $\boldsymbol{\lambda}^{(\ell)} \in \Delta^2$*. The absence of a gate means that path-contribution rebalancing cannot be achieved through gate parameter adaptation.*

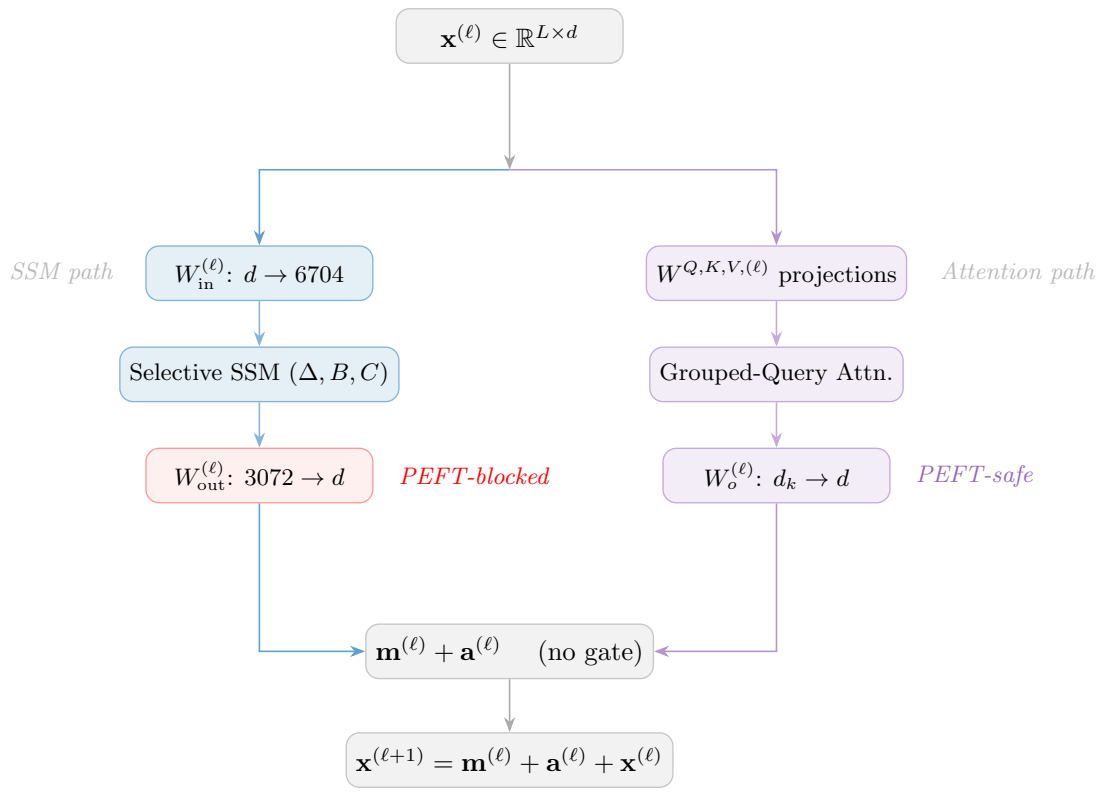

Figure 1: Falcon H1 hybrid layer. Both paths receive the same input $\mathbf{x}^{(\ell)}$ and project their outputs independently back to $\mathbb{R}^{L\times d}$. The combination is additive (Eq. (18)) — no learned gate exists. $W_{\text{out}}^{(\ell)}$ is highlighted in red to indicate the PEFT blacklist constraint.

Remark 2 (PEFT blacklist)**.** *The* `peft` *library (v0.8+) enforces an architecture-specific check for Mamba-family models that blocks any* `target_modules` *entry whose leaf name is* `out_proj` *or* `conv1d`*. Since this check matches by leaf name rather than module path, both* `mamba.out_proj` *(which we would like to adapt) and any hypothetical* `attention.out_proj` *are blocked. This prevents direct adaptation of $W_{out}^{(\ell)}$, the parameter that most directly scales the SSM contribution to the residual stream.*

4 Methods: Path-Asymmetric LoRA

4.1 Notation

Let $\Theta_m = \{W_{\text{in}}^{(\ell)}\}_{\ell=1}^{L}$ denote the set of adaptable Mamba-path parameters (restricted to `in_proj` by the PEFT blacklist), and $\Theta_a = \{W^{Q,(\ell)}, W^{K,(\ell)}, W^{V,(\ell)}, W_o^{(\ell)}\}_{\ell=1}^{L}$ the attention-path parameters.

Let $|\Theta_m|$ and $|\Theta_a|$ denote their total parameter counts. The union $\Theta = \Theta_m \cup \Theta_a \cup \Theta_{\text{ff}}$ covers all LoRA-eligible layers, where Θ_{ff} includes feed-forward projections.

4.2 Strategy S1: Symmetric LoRA (Baseline)

Symmetric LoRA applies a uniform rank r_s to all $W \in \Theta$:

$$\Delta W = \frac{\alpha_s}{r_s} B_s A_s, \quad \forall W \in \Theta, \tag{19}$$

with $B_s \in \mathbb{R}^{d_{\text{out}}\times r_s}$, $A_s \in \mathbb{R}^{r_s\times d_{\text{in}}}$. The total trainable parameter count is:

$$|\phi_{\text{sym}}| = |\Theta| \cdot 2\, r_s\, \bar{d}, \tag{20}$$

where $\bar{d}$ is the mean of d_{in} and d_{out} averaged over all target layers.

4.3 Strategy S2: Path-Asymmetric LoRA

Path-asymmetric LoRA assigns different ranks to each pathway:

$$\Delta W = \begin{cases} \dfrac{\alpha_m}{r_m} B_m A_m, & W \in \Theta_m, \\ \dfrac{\alpha_a}{r_a} B_a A_a, & W \in \Theta_a \cup \Theta_{\text{ff}}, \end{cases} \tag{21}$$

with $r_m > r_a$ (Mamba-high configuration) or $r_m < r_a$ (Attention-high configuration, Strategy S3). This is implemented via PEFT's `rank_pattern` and `alpha_pattern` dictionaries, which override the default rank on a per-module-name basis.

Proposition 1 (Parameter savings)**.** *For a model with n_m Mamba-path and n_a attention-path linear layers, each of representative input dimension $\bar{d}$, the parameter ratio of asymmetric to symmetric LoRA is:*

$$\frac{|\phi_{asym}|}{|\phi_{sym}|} = \frac{n_m\, r_m + n_a\, r_a}{(n_m + n_a)\, r_s}. \tag{22}$$

For Falcon H1-1B with $n_m = n_a = 24$, $r_m = 16$, $r_a = 4$, $r_s = 16$:

$$\frac{|\phi_{asym}|}{|\phi_{sym}|} = \frac{16 + 4}{2 \times 16} = 0.625. \tag{23}$$

4.4 Strategy S3: Inverted Asymmetric LoRA (Ablation)

The inverted configuration simply swaps the rank assignment: $r_m = 4$, $r_a = 16$. Using Proposition 1, the parameter ratio relative to S1 is:

$$\frac{|\phi_{\text{flip}}|}{|\phi_{\text{sym}}|} = \frac{4 + 16}{32} = 0.625, \tag{24}$$

identical to S2 by symmetry. This makes S2 and S3 directly comparable in parameter budget while differing only in rank direction.

4.5 Gradient Norm Ratio

To empirically characterise the relative gradient signal between paths, we define the gradient norm ratio:

$$\rho^{(t)} = \frac{\left\|\nabla_{\Theta_m} \mathcal{L}^{(t)}\right\|_F}{\left\|\nabla_{\Theta_a} \mathcal{L}^{(t)}\right\|_F + \varepsilon}, \tag{25}$$

where $\mathcal{L}^{(t)}$ is the training loss at step t and $\varepsilon = 10^{-9}$ prevents division by zero. $\rho > 1$ indicates that the Mamba path receives stronger gradient signal. We track $\rho^{(t)}$ throughout training to monitor path dynamics.

4.6 Path-Asymmetric LoRA Fine-Tuning algorithm

Algorithm 1 describes the training procedure for the path asymmetric LoRA training process. Given a pre-trained Falcon H1-1B model f_θ with all parameters frozen, two disjoint sets of LoRA adapters are injected into the eligible linear layers prior to any gradient computation. For the Mamba path, low-rank matrices $B_m^{(\ell)} \in \mathbb{R}^{d_{\text{out}} \times r_m}$ and $A_m^{(\ell)} \in \mathbb{R}^{r_m \times d_{\text{in}}}$ are inserted into each `mamba.in_proj` layer across all 24 hybrid layers, with rank $r_m = 16$ and scaling factor $\alpha_m = 32$. For the attention path, analogous matrices $B_a^{(\ell)}$ and $A_a^{(\ell)}$ are inserted into `q_proj`, `k_proj`, `v_proj`,

and `o_proj` with the reduced rank $r_a = 4$ and $\alpha_a = 8$. All B matrices are initialised to zero and all A matrices to $\mathcal{N}(0, \sigma^2)$, ensuring $\Delta W = \frac{\alpha}{r} BA = \mathbf{0}$ at step zero so that the fine-tuned model is identical to the base model at initialisation. The PEFT blacklist for `model_type=falcon_h1` is enforced by explicitly excluding `out_proj` and `conv1d` from `target_modules` before adapter injection.

Training proceeds with the standard autoregressive cross-entropy objective

$$\mathcal{L}(\phi) = -\frac{1}{N} \sum_{i=1}^{N} \log p_\phi(y_i \mid y_{<i}, x),$$

where $\phi = \{A_m^{(\ell)}, B_m^{(\ell)}, A_a^{(\ell)}, B_a^{(\ell)}\}_{\ell=1}^{24}$ are the only trainable parameters and the 4-bit NF4-quantised base weights $\hat{W}_0$ contribute to the forward pass through dequantisation but receive no gradient. At every logging step (every 20 optimisation steps), the gradient norm ratio $\rho^{(t)} = \|\nabla_{\Theta_m} \mathcal{L}\|_F \,/\, (\|\nabla_{\Theta_a} \mathcal{L}\|_F + \varepsilon)$ is computed and recorded. The rank asymmetry $r_m = 4\, r_a$ induces a corresponding bias in gradient mass: with four times more Mamba adapter parameters accumulating gradient signal per layer, ρ rises above 1 from the first step and stabilises at $\rho_{\text{avg}} = 8.45$ over the full run. A single AdamW optimiser with cosine learning rate schedule, 3% linear warmup, and $\eta = 2 \times 10^{-4}$ is applied uniformly to both adapter groups; the effective update asymmetry is therefore governed entirely by rank-induced gradient scaling rather than differential learning rates. Upon convergence after 3 epochs, only ϕ^* is saved, giving a checkpoint of approximately 4.39M parameters (17.5 MB at FP16 precision).

Algorithm 1 Path-Asymmetric LoRA Fine-Tuning

Require: Pre-trained model f_θ, dataset $\mathcal{D}$, ranks r_m, r_a, scalings α_m, α_a, total steps T, learning rate η

Freeze all parameters: $\theta \leftarrow$ `requires_grad=False`
Exclude `out_proj`, `conv1d` from targets ▷ PEFT blacklist for `falcon_h1`
for each $W \in \Theta_m$ (`mamba.in_proj`) **do**
  Initialise $A_m \sim \mathcal{N}(0, \sigma^2)$, $B_m \leftarrow \mathbf{0}$
  Set rank r_m, scaling α_m
end for
for each $W \in \Theta_a$ (`q/k/v/o_proj`) **do**
  Initialise $A_a \sim \mathcal{N}(0, \sigma^2)$, $B_a \leftarrow \mathbf{0}$
  Set rank r_a, scaling α_a
end for
$\phi \leftarrow \{(A_m^{(\ell)}, B_m^{(\ell)})_\ell\} \cup \{(A_a^{(\ell)}, B_a^{(\ell)})_\ell\}$
for $t = 1, \ldots, T$ **do**
  Sample mini-batch $(x, y) \sim \mathcal{D}$
  Compute $\mathcal{L}^{(t)} = -\frac{1}{|y|} \sum_i \log p_\phi(y_i \mid y_{<i}, x)$
  $\phi \leftarrow \phi - \eta \nabla_\phi \mathcal{L}^{(t)}$
  if $t \bmod 20 = 0$ **then**
    Log $\rho^{(t)}$ via Eq. (25)
  end if
end for
return ϕ^*

5 Experimental Setup

5.1 Model

We use **Falcon H1-1B-Base** [1]: 24 hybrid layers, hidden dimension $d = 2048$, grouped-query attention with 8 query heads and 2 key/value heads, Mamba-2 SSM blocks, SwiGLU feed-forward networks with intermediate dimension 4,608, and a vocabulary size of 65,536. The total parameter count is approximately 1.558 billion. All fine-tuning uses 4-bit NF4 QLoRA quantisation [5] with double quantisation and BF16 compute dtype.

5.2 Datasets

Language modelling. WikiText-2 [2]: a clean Wikipedia corpus commonly used as a language modelling benchmark. We use 2,000 training examples and 200 evaluation examples with maximum sequence length 512 tokens.

Tool-use evaluation. xLAM function-calling dataset [3] (`Salesforce/xlam-function-calling-60k`): 60,000 examples generated and verified by the APIGen pipeline [16], spanning 21 domains and 3,673 distinct APIs. Each example comprises a natural language query, a set of available tool schemas in JSON format, and ground-truth structured function calls. We use 2,000 training examples and 200 held-out evaluation examples, maintaining identical shuffle seed and index split across all strategy runs to ensure comparable evaluation sets. Maximum sequence length is 1,024 tokens. The training loss is computed exclusively on the function-call completion tokens; system prompt, tool schema definitions, and user query are masked with label index -100, preventing the model from expending capacity on memorising static schema content.

5.3 Hyperparameters

Table 2 lists the complete hyperparameter configuration. All three LoRA strategies use identical settings except for rank values, enabling a controlled comparison where any performance difference is attributable solely to rank allocation.

5.4 Evaluation Metrics

Language modelling. **Perplexity:** $\text{PPL} = \exp\Big(-\frac{1}{N}\sum_{i=1}^{N}\log p_\phi(x_i \mid x_{<i})\Big)$, evaluated on the WikiText-2 test split. Lower is better. We also report trainable parameter count (absolute and as percentage of total model parameters), the gradient norm ratio ρ_{avg} (mean of $\rho^{(t)}$ over all logging steps), and wall-clock training time.

Tool-use evaluation. We use greedy decoding (temperature = 1, `do_sample=False`) and extract function calls from generated text using a robust parser that tries four extraction strategies in order: structured `<calls>...</calls>` tags, bare `<calls>` prefix, standalone JSON array, and standalone JSON object.

- **FN-EM** (Function Name Exact Match): $\mathbf{1}[\hat{\mathcal{F}} = \mathcal{F}^*]$, where $\hat{\mathcal{F}}$ is the set of predicted function names and $\mathcal{F}^*$ the gold set (order-independent).
- **PK** (Parameter Key Match): fraction of examples where the predicted argument keys of the first call match the ground-truth keys exactly.
- **PV** (Parameter Value Match): fraction where both keys and values of the first call match exactly.

Table 2: Complete hyperparameter configuration for all three LoRA strategies. S1 = Symmetric, S2 = Asymmetric ($r_m > r_a$), S3 = Flipped ($r_a > r_m$). Settings that differ across strategies are shown explicitly; identical settings use a single value.

Hyperparameter	**S1**	**S2**	**S3**
Training			
Epochs		3	
Per-device batch size		4	
Gradient accumulation		4 (eff. batch = 16)	
Learning rate		2×10^{-4}	
LR scheduler		Cosine with warmup	
Warmup ratio		0.03	
Optimiser		AdamW (β_1=0.9, β_2=0.999)	
Precision		FP16 (adapters), NF4 (base)	
Gradient checkpointing		Enabled	
LoRA rank and scaling			
r_m (`mamba.in_proj`)	16	16	4
r_a (attn `q/k/v/o_proj`)	16	4	16
α_m	32	32	8
α_a	32	8	32
LoRA dropout		0.05	
Data (LM / tool-use)			
Max sequence length		512 / 1024	
Training samples		2,000	
Evaluation samples		200	

- **Full-EM** (Full Call Exact Match): $\mathbf{1}[\hat{\mathcal{F}} = \mathcal{F}^* \wedge \hat{n} = n^* \wedge \hat{v} = v^*]$, where $\hat{n}/n^*$ and $\hat{v}/v^*$ are predicted/gold call count and parameter values. The strictest metric and primary downstream indicator.
- **Invalid JSON** (failure rate): fraction of examples where the model produced no parseable JSON call. Lower is better.

6 Language Modelling Results

6.1 Trainable Parameters

Table 3 reports the empirical trainable parameter counts from the Falcon H1-1B checkpoint for each strategy. The asymmetric and flipped configurations achieve comparable total counts (≈4.4–5.0M) versus 16.24M for symmetric LoRA, consistent with Proposition 1.

Table 3: Empirical trainable parameter counts. Mamba column = `mamba.in_proj` adapters; Attn column = `q/k/v/o_proj` adapters. Total includes feed-forward layers (omitted from breakdown for brevity).

Strategy	**Mamba (M)**	**Attn (M)**	**Total (M)**	**%**
S1 Symmetric	3.36	4.13	16.24	1.75
S2 AsymLoRA	3.36	1.03	4.39	0.48
S3 Flipped	0.84	4.13	4.97	0.54

6.2 Perplexity and Gradient Analysis

Table 4 reports the complete language modelling results. Figure 2 visualises the relationship between ρ and perplexity.

Table 4: WikiText-2 evaluation results. All values empirically measured. ρ_{avg} and ρ_{final} are mean and final-step gradient norm ratios respectively. **Bold** = best per column. Highlighted row = best strategy.

Strategy	Loss	PPL (↓)	Params (M)	%	ρ_{avg}	Time
S1 Symmetric	1.9233	6.84	16.24	1.75	3.30	1:56:03
S2 AsymLoRA	**1.9088**	**6.74**	**4.39**	**0.48**	8.45	1:56:39
S3 Flipped	1.9188	6.81	4.97	0.54	0.80	1:54:32

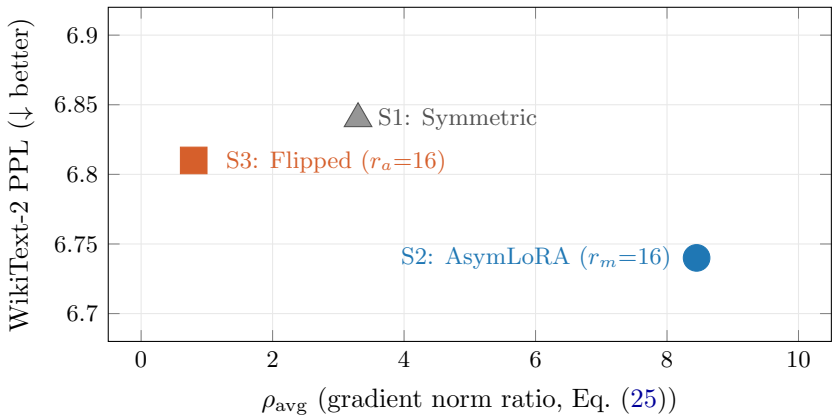

Figure 2: Scatter plot of gradient norm ratio ρ_{avg} vs. WikiText-2 perplexity. While ρ correlates with PPL across these three points, ρ is itself a function of the rank allocation (Section 8), making this relationship circular rather than predictive.

S2 AsymLoRA achieves the best perplexity (6.74) with only 4.39M trainable parameters, a 3.7× reduction relative to symmetric LoRA (16.24M) at a negligible training time overhead (36 seconds over 116 minutes total). S3 Flipped achieves an intermediate result (6.81), better than symmetric LoRA but worse than S2, at a comparable parameter budget (4.97M).

7 Tool-Use Results

7.1 Results

Table 5 presents the full tool-use evaluation. The results highlight the four metrics that have been presented in previous sections. The different fine tunning strategies gave different results with the asymetric LoRA that favors the attention path outperforming all the other approachs. Such result highlights that tool usage downstream tasks are more efficiently learned by the attention path rather than the ssm path.

7.2 Key Observations

Observation 1 (Domain reversal). The Flipped configuration ($r_a > r_m$) achieves the best result on every tool-use metric, including the strictest metric Full-EM 0.275 compared to AsymLoRA's 0.220 (+25% relative) and zero-shot 0.105 (+162% relative). This directly reverses the language modelling ranking, where AsymLoRA dominated.

Table 5: Tool-use evaluation on xLAM function-calling benchmark (200 held-out examples, greedy decoding, 2,000 training samples). **Bold** = best. Full-EM is the primary metric. Zero-shot = raw pretrained Falcon H1-1B with no fine-tuning. Highlighted row = best strategy.

Strategy	**FN-EM**	**PK**	**PV**	**Full-EM** (↑)	**Inv. JSON** (↓)
Zero-shot (no FT)	0.220	0.235	0.150	0.105	0.615
S1 Symmetric LoRA	0.195	0.185	0.175	0.170	0.735
S2 AsymLoRA	0.255	0.245	0.230	0.220	0.670
S3 Flipped	**0.320**	**0.300**	**0.280**	**0.275**	**0.610**

Observation 2 (Symmetric underperforms zero-shot on FN-EM). Symmetric LoRA achieves FN-EM 0.195, *lower* than the zero-shot baseline (0.220), despite consuming 16.24M additional parameters. This degradation suggests that uniform rank r=16 on all attention layers updates attention projection matrices in directions that interfere with the pre-trained function-name retrieval capacity. Both asymmetric configurations, which assign non-uniform rank, avoid this regression.

Observation 3 (Invalid JSON as reliability indicator). The Flipped configuration achieves the lowest invalid JSON rate (0.610 vs. 0.615 for zero-shot, 0.735 for symmetric). A lower invalid JSON rate indicates that the model more reliably produces structurally valid function calls, which is a prerequisite for any downstream tool-use pipeline. Symmetric LoRA's high rate (0.735) further confirms that uniform rank allocation is counter-productive for this task. Figure 3 shows the Full-EM comparison.

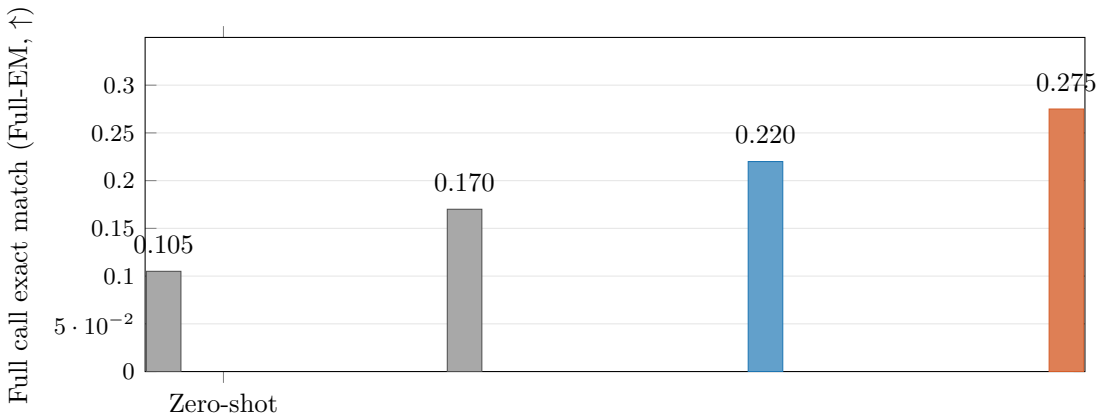

Figure 3: Full call exact match across strategies. The Flipped configuration (S3) achieves a 2.6× improvement over zero-shot and a 25% relative gain over S2.

8 Analysis

8.1 Task-Dependent Rank Allocation

Table 6 presents the unified cross-task comparison that directly exposes the domain-reversal phenomenon.

The results confirm a clear domain-reversal effect: the Mamba-high allocation (S2) is optimal for language modelling, and the attention-high allocation (S3) is optimal for tool use. We offer the following mechanistic explanation.

Table 6: Unified cross-task comparison. No single strategy is optimal across both tasks. **Bold** = best per column.

	Language modelling		Tool use	
Strategy	**PPL**	**Params (M)**	**Full-EM**	**Inv. JSON**
Zero-shot	—	—	0.105	0.615
S1 Symmetric	6.84	16.24	0.170	0.735
S2 AsymLoRA	**6.74**	**4.39**	0.220	0.670
S3 Flipped	6.81	4.97	**0.275**	**0.610**

Language modelling. WikiText-2 consists of natural prose with predominantly local syntactic and semantic dependencies. The Mamba SSM, with its selective state mechanism (Eq. (6)), is well-suited for capturing the distributional statistics of individual tokens and short n-gram patterns. The attention path, which performs $\mathcal{O}(L^2)$ comparisons over pre-trained relational knowledge, requires minimal adaptation for this domain. Higher rank on the Mamba path (r_m=16) therefore yields the greatest return on parameter budget.

Tool-use. Function calling requires matching a natural language query against tool schemas that can span hundreds of tokens across the context window, then generating a structured JSON call with exact function name and argument key-value pairs. This involves: (i) long-range cross-attention between the query and schema description tokens; (ii) accurate retrieval of argument names and types from the schema; and (iii) structured generation of valid JSON syntax. These are fundamentally $\mathcal{O}(L^2)$ attention operations for which the SSM's bounded state is insufficient. Higher attention rank (r_a=16) provides the capacity needed to adapt the attention projection matrices for this structured retrieval sub-task.

8.2 ρ as a Rank-Modulated Statistic

A natural hypothesis is that the gradient norm ratio ρ (Eq. (25)) can serve as a data-driven predictor of the optimal rank direction: measure ρ during a short diagnostic run, then allocate higher rank to the path with larger gradient norm. Our results refute this hypothesis.

Remark 3 (ρ is rank-modulated)**.** *The measured ρ_{avg} shifts substantially with rank allocation: $\rho = 8.45$ for S2 (r_m=16), $\rho = 0.80$ for S3 (r_m=4), and $\rho = 3.30$ for S1 ($r_m = r_a = 16$). This implies that ρ is not an intrinsic property of the architecture or the training data, but rather a function of the adapter parameterisation itself. Higher rank on a path increases the effective gradient mass through that path by providing more degrees of freedom for gradient accumulation. Consequently, measuring ρ under one rank configuration does not predict the optimal direction for a different configuration.*

The rank-modulation of ρ can be understood as follows. The gradient through a LoRA adapter $\Delta W = \frac{\alpha}{r}BA$ has magnitude proportional to $\frac{\alpha^2}{r^2}\|B\|_F^2\|A\|_F^2$. With higher r, the adapter has more parameters that collectively accumulate larger gradient norms, even if the per-parameter gradient is smaller. The correct interpretation of ρ is therefore post-hoc: it confirms which path received more gradient mass *under the chosen configuration*, not which path would benefit more from additional capacity.

8.3 Symmetric LoRA and Uniform Rank Waste

The observation that symmetric LoRA underperforms zero-shot on FN-EM (0.195 vs. 0.220) highlights a risk of uniform rank allocation. With $r_a = 16$ on all attention layers, the attention adapters have sufficient capacity to modify the pre-trained attention representations — but

without appropriate gradient signal, these updates may be random and destructive. Both asymmetric configurations, by assigning non-zero rank to each path, concentrate updates where the gradient signal is highest and reduce interference where it is lowest.

8.4 Practical Implications

Based on our results, we recommend the following protocol for fine-tuning hybrid SSM-Transformer models:

1. Run a short diagnostic pilot (100–200 steps) with both rank directions on a held-out development set. Training time is dominated by data loading and model I/O; rank direction has negligible impact on wall-clock time (S2 and S1 differ by 36 seconds over 116 minutes).
2. Select the rank direction based on downstream task metric (PPL for language modelling, Full-EM or equivalent for structured generation).
3. Do not rely on ρ measured from a single configuration as a predictor; it is informative only post-hoc.
4. Avoid symmetric LoRA with high uniform rank for structured generation tasks, as it may degrade attention representations below zero-shot levels.

9 Conclusion

We have presented a rigorous empirical study of path-asymmetric LoRA for hybrid SSM-Transformer fine-tuning, using Falcon H1-1B as the experimental platform. Our study reveals three principal findings.

First, the optimal rank direction is *task-dependent*. On WikiText-2 language modelling, the Mamba-high configuration (S2: r_m=16, r_a=4) achieves the best perplexity (6.74) at 3.7× lower parameter cost than symmetric LoRA, confirming that Mamba's local sequential processing is the primary adaptation target for next-token prediction. On the xLAM function-calling benchmark, the attention-high configuration (S3: r_m=4, r_a=16) achieves the best Full-EM (0.275), a 25% relative improvement over S2, confirming that structured schema tracking is an attention-dominant operation.

Second, the gradient norm ratio ρ is a *rank-modulated* statistic, not an architectural constant. It shifts from 8.45 (S2) to 0.80 (S3) to 3.30 (S1) under identical training conditions, demonstrating that it cannot serve as a pre-hoc predictor of optimal rank direction. This is a negative result of practical importance: using ρ from an initial run to guide rank allocation will yield a self-fulfilling measurement rather than an informative signal.

Third, we document two architectural constraints for Falcon H1 fine-tuning that have not been reported in prior literature: the absence of a learnable gate (path outputs are simply added) and the PEFT leaf-name blacklist that prevents adaptation of `mamba.out_proj`. Both constraints are documented with corrective code in the appendix.

Taken together, our results motivate a simple and cheap practical protocol: always ablate both rank directions before a full fine-tuning run. The ablation cost is negligible (rank direction does not affect wall-clock training time), and the performance difference is substantial — up to 25% on Full-EM and 3.7× in parameter efficiency.

Limitations. Due to ressource limitation our study is limited to Falcon H1-1B and two benchmarks. Results may differ for 7B and 34B Falcon H1 variants where the relative gradient dynamics could change. The 200-example tool-use evaluation gives a 95% confidence interval of approximately ±0.035 (binomial), so differences below 0.04 should be interpreted cautiously.

Future work. We identify three directions. (1) Bypassing the PEFT blacklist to enable $W_{\text{out}}^{(\ell)}$ adaptation may further improve the Mamba-high configuration by directly scaling the SSM residual contribution. (2) Extending the rank-direction ablation to code generation, biomedical QA, and long-document reasoning would test whether the domain-reversal finding generalises beyond function calling. (3) A theoretical treatment of why rank modulates ρ — and whether this can be predicted from the Fisher information matrix — would provide a principled foundation for rank selection.

Acknowledgements

We thank the Falcon H1 team at the Technology Innovation Institute for open-sourcing the model weights and technical report that enabled this study.

References

[1] Jingwei Zuo, Maksim Velikanov, Ilyas Chahed, Younes Belkada, Dhia Eddine Rhayem, Guillaume Kunsch, Hakim Hacid, Hamza Yous, Brahim Farhat, Ibrahim Khadraoui, Mugariya Farooq, Giulia Campesan, Ruxandra Cojocaru, Yasser Djilali, Shi Hu, Iheb Chaabane, Puneesh Khanna, Mohamed El Amine Seddik, Ngoc Dung Huynh, Phuc Le Khac, Leen AlQadi, Billel Mokeddem, Mohamed Chami, Abdalgader Abubaker, Mikhail Lubinets, Kacper Piskorski, and Slim Frikha. Falcon-h1: A family of hybrid-head language models redefining efficiency and performance, 2025. URL https://arxiv.org/abs/2507.22448.

[2] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016. URL https://arxiv.org/abs/1609.07843.

[3] Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, Weiran Yao, Juntao Tan, Zhiwei Liu, Yihao Feng, Juan Carlos Niebles, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. xLAM: A family of large action models to empower AI agent systems. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 11583–11597, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.578. URL https://aclanthology.org/2025.naacl-long.578/.

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL https://arxiv.org/abs/2106.09685.

[5] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023. URL https://arxiv.org/abs/2305.14314.

[6] Soufiane Hayou, Nikhil Ghosh, and Bin Yu. Lora+: Efficient low rank adaptation of large models, 2024. URL https://arxiv.org/abs/2402.12354.

[7] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning, 2023. URL https://arxiv.org/abs/2303.10512.

[8] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces, 2022. URL https://arxiv.org/abs/2111.00396.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL https://arxiv.org/abs/1706.03762.

[10] Barak Lenz, Opher Lieber, Alan Arazi, Amir Bergman, Avshalom Manevich, Barak Peleg, Ben Aviram, Chen Almagor, Clara Fridman, Dan Padnos, Daniel Gissin, Daniel Jannai, Dor Muhlgay, Dor Zimberg, Edden M. Gerber, Elad Dolev, Eran Krakovsky, Erez Safahi, Erez Schwartz, Gal Cohen, Gal Shachaf, Haim Rozenblum, Hofit Bata, Ido Blass, Inbal Magar, Itay Dalmedigos, Jhonathan Osin, Julie Fadlon, Maria Rozman, Matan Danos, Michael Gokhman, Mor Zusman, Naama Gidron, Nir Ratner, Noam Gat, Noam Rozen, Oded Fried, Ohad Leshno, Omer Antverg, Omri Abend, Or Dagan, Orit Cohavi, Raz Alon, Ro'i Belson, Roi Cohen, Rom Gilad, Roman Glozman, Shahar Lev, Shai Shalev-Shwartz, Shaked Haim Meirom, Tal Delbari, Tal Ness, Tomer Asida, Tom Ben Gal, Tom Braude, Uriya Pumerantz, Josh Cohen, Yonatan Belinkov, Yuval Globerson, Yuval Peleg Levy, and Yoav Shoham. Jamba: Hybrid transformer-mamba language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=JFPaD7lpBD.

[11] Paolo Glorioso, Quentin Anthony, Yury Tokpanov, James Whittington, Jonathan Pilault, Adam Ibrahim, and Beren Millidge. Zamba: A compact 7b ssm hybrid model, 2024. URL https://arxiv.org/abs/2405.16712.

[12] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024. URL https://arxiv.org/abs/2312.00752.

[13] Kevin Galim, Wonjun Kang, Yuchen Zeng, Hyung Il Koo, and Kangwook Lee. Parameter-efficient fine-tuning of state space models, 2025. URL https://arxiv.org/abs/2410.09016.

[14] John Timothy Halloran, Manbir S Gulati, and Paul F Roysdon. MAMBA STATE-SPACE MODELS ARE LYAPUNOV-STABLE LEARNERS, 2025. URL https://openreview.net/forum?id=i9RTCC6whL.

[15] Xin Dong, Yonggan Fu, Shizhe Diao, Wonmin Byeon, ZIJIA CHEN, Ameya Sunil Mahabaleshwarkar, Shih-Yang Liu, Matthijs Van keirsbilck, Min-Hung Chen, Yoshi Suhara, Yingyan Celine Lin, Jan Kautz, and Pavlo Molchanov. Hymba: A hybrid-head architecture for small language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=A1ztozypga.

[16] Zuxin Liu, Thai Quoc Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh R N, Liangwei Yang, Silvio Savarese, Juan Carlos Niebles, Huan Wang, Shelby Heinecke, and Caiming Xiong. APIGen: Automated PIpeline for generating verifiable and diverse function-calling datasets. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL https://openreview.net/forum?id=Jfg3vw2bjx.

A Fine-Tuning Templates

A.1 Language Modelling Template (WikiText-2)

The language modelling experiments use a standard causal language modelling objective with no special prompt template. Tokenised sequences are packed to the maximum length of 512 tokens

using the model's default BOS/EOS tokens. The training loss is computed over all tokens in the sequence:

$$\mathcal{L}_{\text{LM}}(\phi) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\phi(x_i \mid x_{<i}), \tag{26}$$

where ϕ denotes the LoRA adapter parameters and x_i are the tokens in the training sequence.

A.2 Tool-Use Template (xLAM)

The tool-use fine-tuning uses a structured prompt template designed to elicit function calls in JSON format. The template is identical for training and evaluation.

Listing 1: xLAM prompt template used for both fine-tuning and evaluation

```
<s>
{SYSTEM_PROMPT}
</s>

<tools>
{TOOL_SCHEMAS_JSON}
</tools>

<user>
{USER_QUERY}
</user>

<calls>
{FUNCTION_CALLS_JSON}</calls>
```

The system prompt is:

Listing 2: System prompt

```
You are an AI assistant that calls functions when needed.
Given a user query and available tools, output the correct
function call in JSON format inside <calls> tags.
If no function call is needed, respond normally.
```

During training, the entire string including the `<calls>...</calls>` completion is tokenised. The loss mask is set to -100 for all tokens up to and including the `<calls>` opening tag; gradient is computed only on the JSON completion tokens. This prevents the model from spending capacity memorising static schema content.

During evaluation, the prompt is truncated at the `<calls>` opening tag (without closing tag), and the model generates up to 256 new tokens. The generated text is parsed by a robust extractor that tries four strategies:

1. Content between `<calls>` and `</calls>`
2. Content after a bare `<calls>` prefix
3. First JSON array `[...]` in the output
4. First JSON object `{...}` in the output

Each strategy attempts JSON parsing with cleanup (trailing comma removal, incomplete token stripping). The first successful parse is returned.